

PharmaGenDSS: A Mini Clinical Genomics Decision Support System for Variant-Informed Prescribing

John Marquel Revilla

Department of Industrial Engineering & Operations Research
University of the Philippines Diliman
Quezon City, Philippines
jprevilla1@up.edu.ph

ABSTRACT

Adverse drug reactions account for roughly 6.5% of adult hospital admissions, and a big chunk of that variability comes down to inherited differences in how people process drugs. The science addressing this is reasonably mature—CPIC and DPWG between them already cover more than a hundred clinically actionable gene–drug pairs—but the guidelines aren't being used routinely at the bedside. PharmGKB curates the underlying evidence, DrugBank curates the drug records, and CPIC and DPWG translate both into prescribing recommendations. None of these are prescription-aware out of the box.

This paper proposes **PharmaGenDSS**, a small, single-tenant decision-support system that pulls those four resources together into one citation-backed query interface. The intended users are prescribing clinicians, clinical pharmacists, bioinformatics curators, and pharmacogenomics researchers. The proposal follows Sommerville's software-engineering frameworks throughout: requirements are gathered through focused ethnography and structured scenarios; twelve functional requirements and three categories of non-functional requirements are specified; the system is modelled with contextual, structural, and behavioral views; the architecture is layered with a Repository pattern and a pipe-and-filter sub-architecture for the curator's data refresh; and the implementation identifies six principal classes, six design patterns, and the four families of implementation issues.

The architecture is built to honor the share-alike obligations of PharmGKB's CC-BY-SA license and the academic-license boundaries of DrugBank, primarily through a role-based redaction layer at the application boundary. Deployment is via Docker Compose, on a single tenant. The methodology is Agile with a DevOps complement, with corresponding documentation set.

*J.M. Revilla is a third year Master of Engineering student (specializing in Analytics Systems Engineering) of the Department of Industrial Engineering and Operations Research, University of the Philippines Diliman, Quezon City Philippines 1101 (e-mail: jprevilla1@up.edu.ph).

Keywords: pharmacogenomics, decision support, clinical genomics, software architecture, requirements engineering, Sommerville, layered architecture, Repository, pipe-and-filter, CPIC, DPWG, PharmGKB, DrugBank.

1 Project Background

1.1 Motivation

Two patients on the same drug at the same dose can have very different outcomes. One responds well, the next has an adverse reaction, the third sees no benefit. A large part of that variation is inherited: the genes that handle drug metabolism, drug transport, and immune response differ from person to person. The Pirmohamed *et al.* prospective study of 18,820 patients found adverse drug reactions cause around 6.5% of adult hospital admissions [2], and the contribution of inherited variation in genes like CYP2C9, CYP2C19, CYP2D6, SLCO1B1, and HLA-B is well-documented [3].

The science isn't really the problem here. CPIC has published more than thirty peer-reviewed prescribing guidelines that translate genotype data into actual recommendations, with evidence grades from Level 1A through Level 4 [5]. DPWG (the Dutch counterpart, maintained by the Royal Dutch Pharmacists Association) publishes complementary recommendations that are integrated directly into Dutch pharmacy software [6]. Between the two, more than a hundred clinically actionable gene–drug pairs are covered.

Despite that, point-of-care adoption is uneven. A clinician who wants to consult pharmacogenomic evidence at the moment of prescribing has to traverse multiple unconnected resources: variant-level evidence in PharmGKB [3], drug-level metadata in DrugBank [4], guideline recommendations from CPIC and DPWG, and citation lookups in PubMed. Each of these resources is *descriptive*—it tells the clinician what is known. None of them is *prescriptive*—none of them tells the clinician what to do given a specific patient's genetic profile.

1.2 Problem Statement

The problem this project addresses is the integration gap between curated pharmacogenomic evidence and the prescribing decision. Stated more directly: there isn't a widely deployed, vendor-neutral, lightweight system that lets a prescribing clinician ask, in one query, *"given this patient's genetic profile, what should I prescribe?"* and get back a guideline-backed, citation-traceable answer in a clinically reasonable time.

1.3 Project Scope

PharmaGenDSS is positioned as a small, single-tenant, web-accessible decision-support system that brings together PharmGKB, DrugBank, CPIC, and DPWG into one query layer. The MVP supports three driving query types—variant-driven, drug-driven, and gene-driven—through a thin web interface. It surfaces CPIC and DPWG evidence levels along with the citations that back them. It supports four user roles, with appropriate license-aware redaction. And it writes an immutable, anonymized audit-log entry for every query.

A few things are deliberately out of scope for the MVP. Direct integration with hospital Electronic Health Record systems via FHIR Genomics resources [8] is deferred. So is machine-learning-based novel evidence ranking. So is multi-tenant SaaS deployment. The scope is tight on purpose: each of those is a multi-month workstream that doesn't change the proof-of-concept.

1.4 Significance & Positioning

What this proposal contributes isn't the underlying data. PharmGKB, DrugBank, CPIC, and DPWG already do that work, and they do it well. The contribution is the systems-engineering integration layer between them: harmonizing identifiers across sources, layering the CPIC/DPWG recommendations on top of the underlying evidence, applying license-aware redaction by user role, writing an auditable query history, and presenting all of that through an interface shaped to fit a clinician's workflow with a sub-2-second response budget. That's the work the data sources don't do for themselves, and it's what makes the existing science clinically usable.

2 Requirements Engineering & System Model

2.1 Methodology

The methodology is Agile, with a DevOps complement for deployment and continuous integration. Agile fits this project for a few reasons. The requirements will evolve as clinicians give feedback through prototype walkthroughs. There's real value in getting working software in front of users early—a single gene-drug-pair MVP can be demonstrated by sprint two. And the risk profile is moderate, not the high-risk regime that justifies Spiral. The DevOps side handles the host-target reproducibility issue from Sommerville, mainly through containerized deployment via

Docker Compose and continuous integration that runs on every push.

2.2 Requirements Elicitation

Elicitation rests on four techniques:

1. **Focused ethnography**—observation of prescribing clinicians during ward rounds, medication reconciliation, and discharge planning, to see when and how drug-information sources are consulted in practice (which isn't always when the textbooks say they should be).
2. **Stories and scenarios**—structured the way Sommerville recommends: *initial assumption* → *normal flow* → *what can go wrong* → *final state*.
3. **Use cases**—three driving interactions, written down formally in the tabular UML form used in Problem Set 2.
4. **Document review**—CPIC and DPWG guidelines, the FDA Table of Pharmacogenomic Biomarkers in Drug Labeling [7], and the FHIR Genomics Implementation Guide [8].

Every iteration runs through Sommerville's five validation checks: validity, consistency, completeness, realism, and verifiability.

2.3 Intended Users

Four user classes have been identified. Each one drives at least one functional requirement.

Persona	Role	Primary Goal	Key Requirements
Prescribing clinician	Physician (internal medicine, oncology, cardiology)	Variant-to-drug lookup at point of care	Fast, ≤ 3-click lookup; clear recommendation; visible evidence levels
Clinical pharmacist	Hospital or community pharmacist	Validate variant–drug pairs and dose adjustments	Full evidence, citation, dose-adjustment guidance
Bioinformatics curator	Pharmacogenomics data steward	Ingest PharmGKB / DrugBank releases; manage versions	Versioned data refresh, dry-run, rollback, audit
Researcher	Pharmacogenomics or clinical-genomics researcher	Aggregate queries by gene, drug class, or population	Bulk export, anonymised audit trail, statistical filters

2.4 Driving Scenario

The driving scenario is structured as follows:

Initial assumption: A 58-year-old patient has just had a coronary procedure. The cardiologist needs to choose an antiplatelet. The patient's CYP2C19 genotype is on file, and it indicates a slow-metabolizer profile for clopidogrel.

Normal flow: The clinician opens PharmaGenDSS, queries by patient identifier, and the system returns ranked drug options. Clopidogrel is flagged red—avoid for slow metabolizers, CPIC Level 1A. Two alternatives are flagged green. The clinician picks one of the alternatives. The system shows supporting CPIC citation and exports a PDF for the chart.

What can go wrong: (a) The variant isn't in PharmGKB. The system explicitly returns "no evidence" rather than a silent empty result, because a silent empty result could be misread as "no concerns." (b) CPIC and DPWG disagree on the recommendation. Both are surfaced side-by-side with their evidence levels. The system never silently picks one. (c) The clinician role isn't licensed to view some DrugBank fields. The user interface hides them, and the redaction is logged.

Final state: A genotype-aware decision is reached in under thirty seconds. An anonymized audit entry records the query, the recommendations shown, and the drug selected.

This single scenario drives most of the functional and non-functional requirements that follow.

2.5 Functional Requirements

Twelve functional requirements have been specified. Two of the highest-impact FRs are written out in the structured form (Function / Description / Inputs / Outputs / Information / Action / Pre-condition / Post-condition); the rest follow the same template, summarized below.

2.5.1 The full FR list

#	Requirement
FR1	The system shall ingest PharmGKB clinical and variant annotations (TSV exports).
FR2	The system shall ingest DrugBank drug records (XML) under academic-license terms.
FR3	The system shall normalize variant identifiers (<i>rsID</i> and HGVS) and drug identifiers (DrugBank ID, RxNorm, ATC).
FR4	The system shall support a variant-driven query : given one or more <i>rsIDs</i> , return ranked drug–gene–variant interactions with their CPIC and DPWG evidence levels.
FR5	The system shall support a drug-driven query : given a drug, return associated genes, variants, and recommendations.
FR6	The system shall support a gene-driven query : given a gene symbol, return all known variants and affected drugs.
FR7	The system shall display evidence provenance (originating CPIC or DPWG guideline plus PubMed

	citations) for every recommendation.
FR8	The system should allow filtering by evidence level, population/ethnicity, and drug class.
FR9	The system shall enforce authentication and role-based access control across the four user classes.
FR10	The system shall provide a curator dashboard that triggers the data refresh, displays ingestion history, and supports rollback to the prior release.
FR11	The system shall write an immutable audit-log entry for every query (timestamp, anonymized user identity, query payload, redaction summary).
FR12	The system should support export of query results in PDF and CSV formats.

2.5.2 FR4—Variant-driven query (detailed specification)

Field	Specification
<i>Description</i>	Return ranked drug–variant interactions for one or more patient variants.
<i>Inputs</i>	List of patient variant identifiers (<i>rsID</i> or HGVS); optional drug-class or population filter; authenticated user context.
<i>Outputs</i>	Structured response listing each {drug, gene, variant, recommendation, evidence level, citations}.
<i>Information</i>	ClinicalAnnotation, EvidenceRecord, and Drug repositories.
<i>Action</i>	The system shall normalise the input, resolve the relevant gene, fetch the matching annotations, rank by evidence level, redact license-restricted fields by user role, and return the response.
<i>Pre-condition</i>	Authenticated user; a current data release is loaded.
<i>Post-condition</i>	An audit-log entry is written; response cached for five minutes.

2.5.3 FR10—Curator data refresh (detailed specification)

Field	Specification
<i>Description</i>	Run a versioned refresh of PharmGKB and DrugBank data, with dry-run and rollback support.
<i>Inputs</i>	Release tag (PharmGKB date and DrugBank version); curator credentials.
<i>Outputs</i>	Job report; new promoted release tag; updated catalog entry.
<i>Information</i>	<i>IngestionJob</i> , <i>EvidenceRecord</i> , <i>ClinicalAnnotation</i> entities.
<i>Action</i>	The system shall fetch the sources, validate the schema, write to a holding-area copy of the database, run integrity checks, and promote atomically. If any check fails, rollback.
<i>Pre-condition</i>	Authenticated curator; previous release tagged in the catalog.
<i>Post-condition</i>	Catalog updated; audit log and downstream notifications dispatched.

2.6 Non-Functional Requirements

NFRs are organised into three categories: Product, Organisational, External:

Product. Most queries shall return in under 2 seconds (95th percentile, against a corpus of about 10,000 clinical annotations). The data-refresh job shall finish in under 15 minutes per monthly release. The system shall maintain at least 99.5% availability during business hours. A clinician shall complete a variant-to-drug lookup in three clicks or fewer (the goal is to be verifiable through usability testing). Encryption: TLS 1.3 in transit, AES-256 at rest, bcrypt-hashed credentials. The variant-to-drug recommendation mapping shall be cross-checked against the published CPIC reference test cases.

Organisational. Development shall follow Git/GitHub flow, semantic versioning, and continuous integration via GitHub Actions on every push. Variant nomenclature shall follow HGVS [9]; drug coding shall follow ATC. The project shall deliver the documentation set prescribed. Automated tests shall cover at least 80% of the application-layer code. JSON output shall be compatible with the FHIR Genomics health-data exchange standard [8].

External. The system shall respect the share-alike obligations of PharmGKB's CC-BY-SA 4.0 license and the academic-license boundaries of DrugBank. Future integration with hospital EHR systems via FHIR Genomics resources is planned. The user interface shall prominently display the disclaimer "decision support—not diagnostic", and the system shall collect minimal personal data. The system's recommendations shall align with the FDA Table of Pharmacogenomic Biomarkers in Drug Labeling, without claiming medical-device status.

2.7 Contextual Model

The contextual model defines what's inside the system boundary and what's outside. PharmaGenDSS sits at the center. The four user classes interact through the user interface. External data source—PharmGKB, DrugBank, CPIC, DPWG, PubMed—sit outside the boundary. A hospital EHR connection is deliberately not shown; that's a Phase-4 deferral, not an oversight.

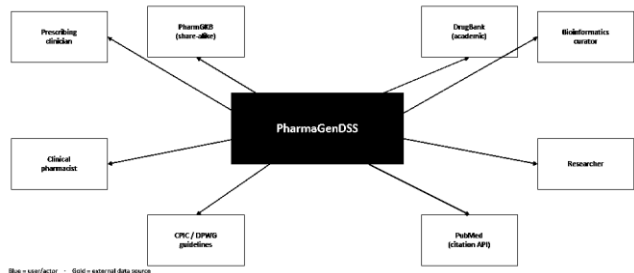


Figure 1. PharmaGenDSS contextual model. User actors flank the system; external data sources sit outside the boundary.

2.8 Use Cases (Interaction Model)

Three use cases drive the design.

UC1—Variant lookup at point of care

Field	Description
Actor	Prescribing clinician
Description	Real-time variant-to-drug lookup at the point of care.
Data	Patient genetic profile; drug recommendations; evidence levels.
Stimulus	Clinician submits a query through the web interface or API.
Response	The system returns ranked recommendations with sources and license-aware redactions. The query is logged.
Comment	Must respond in under 2 seconds (p95). Must not return silently when no evidence exists—must say "no evidence" explicitly.

UC2—Curator data refresh

Field	Description
Actor	Bioinformatics curator
Description	Refresh PharmGKB and DrugBank data, with dry-run and rollback support.
Data	Source files; release tag; integrity-check report.
Stimulus	Curator triggers the data refresh from the dashboard.
Response	The system fetches, validates, holds, integrity-checks, and promotes. Notifications are emitted; the release catalog is updated.
Comment	Must be reversible. The holding area copy of the database is the safety net.

UC3—Aggregate cohort query

Field	Description
Actor	Researcher
Description	Bulk query for genotype prevalence and drug-interaction frequency across a cohort.
Data	Cohort filters (gene, drug class, population); aggregate counts.
Stimulus	Researcher submits an aggregate query with filters.
Response	The system runs the aggregate query against a read-only replica of the database, returns anonymized tabular results, logs the query.
Comment	A minimum cohort size of 50 is enforced to protect individual privacy through statistical-disclosure control.

2.9 Structural Model

Six principal classes: *Variant* carries the standard variant identifiers used to normalize inputs. *Drug* carries the standard drug identifiers—and notice the *redactedView* operation, which is what gates fields by user role for license compliance. *ClinicalAnnotation* is the join entity between *Variant* and *Drug*; it carries the recommendation text and the evidence level. *EvidenceRecord* carries the guideline source and the citation list. *QueryService* is the Façade, which orchestrates the repositories using a Strategy map keyed by query type. *AuditEntry* is the immutable log we discussed in FR11.

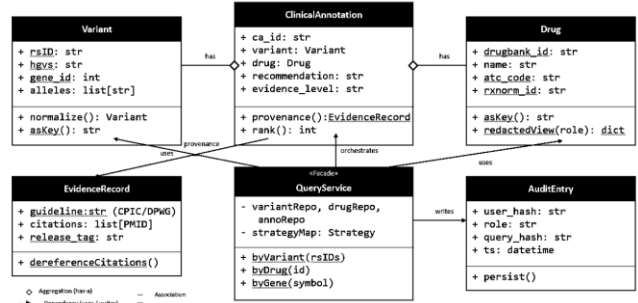


Figure 2. Six principal classes, with + for public visibility and - for private.

A few notes on the *Variant* attributes, since they tend to come up. *rsID* is the reference SNP identifier from NCBI's dbSNP—the stable cross-database identifier. PharmGKB, ClinVar, dbSNP, and most clinical labs all speak *rsID*, so it's the natural input for a variant lookup. *hgvs* is the Human Genome Variation Society (HGVS) nomenclature—it pins down both the position and the exact change. *rsID* alone doesn't tell you the exact alternative base in some cases; HGVS does, and it's what you'd cite in a clinical report. *gene_id* is a foreign key to the *Gene* entity. Storing the gene name on every variant row would be redundant, and it would make gene-driven queries slower. *alleles* is the list of variant forms observed at that position—for example ["G", "A"] for a single-nucleotide variant, or ["*1", "*2", "*3"] for star alleles. Pharmacogenomic recommendations depend on which alleles the patient carries, so we need the full set to match against the CPIC and DPWG genotype-to-phenotype tables.

Drug.redactedView(role) is worth flagging because it's where field-level license compliance happens. The full drug record may include DrugBank fields that aren't licensed for the clinician role, and *redactedView* is the operation that returns a license-respecting copy.

2.10 Behavioral Model

Two behavioral views—a sequence diagram for the variant query (UC1), and a state diagram for the data-refresh job (UC2).

The clinician submits a query through the web user interface. The UI forwards the request to *QueryService* over the API. *QueryService* delegates to the appropriate Strategy handler, which

fans out to the *Variant*, *Drug*, and *ClinicalAnnotation* repositories. The repositories return the matching rows. *QueryService* ranks the results by evidence level, applies role-based redaction internally, and returns a structured response. The UI renders the response to the clinician. The audit entry is written asynchronously, after the response has been sent—so it doesn't slow down the user-visible part.

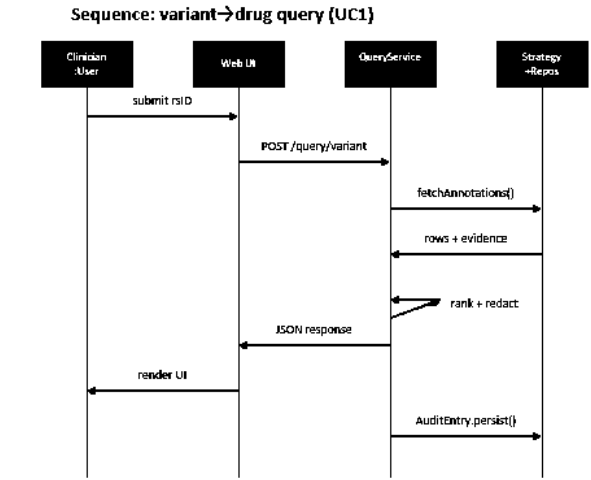


Figure 3. Variant query sequence diagram

On the other hand, the data-refresh job (see Fig. 4) is a small state machine. From *Idle*, a curator-triggered (or scheduler-triggered) event moves it to *Fetching*. Once all source files are downloaded, it moves to *Validating*. If validation passes, it moves to *Loading*, then to *Promoted* once the atomic database swap completes. If validation or any integrity check fails, it transitions to *Rolled-back* instead, and live data is never touched.

State: ETL ingestion job (UC2)

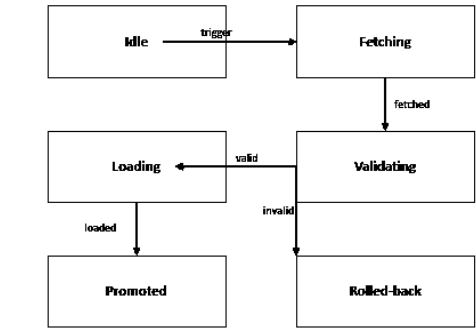


Figure 4. Data-refresh job state diagram. The branch at *Validating* is the safety mechanism—an invalid input routes the job to *Rolled-back* without ever touching live data.

3 System Architecture & Design

3.1 Application Architecture

PharmaGenDSS is a transaction-processing information system. Users submit queries, the system reads from or updates a database in response, and transactions are the unit of work. The standard four-layer information-system shape applies here: *user interface* → *user-communication* → *information retrieval* → *systems database*.

3.2 Architectural Pattern

The proposed architecture (layered + repository) has four logical layers. Each one has a single responsibility: *Presentation*—the web user interface, with different views per user role; *UI Management*—the API server with login, sessions, and auto-generated API docs; *Application Logic*—the query service, the per-query-type strategies, evidence ranking, the redaction policy, and the data-refresh pipeline; and *Data*—repository objects on top of the live database, plus a holding-area copy used during refresh. The Repository pattern sits at the database boundary, so the database engine could be swapped later without touching the rest of the system.

Presentation Layer	Web user interface · different views per role
User Interface Management Layer	API server · login & session · auto-generated API documentation
Application Logic Layer	Query service · per-query-type strategies · evidence ranking · redaction · data-refresh pipeline
Data / Persistence Layer	Repositories on top of the live database + a separate holding-area copy used during refresh

Below is a swimlane diagram showing all four user classes and how each one's workflow flows through the architectural layers. Each user class's workflow enters at Presentation (left) and traverses through UI Management, Application Logic, and Data. The view makes it visible at a glance (e.g. only the Curator engages in the holding-area schema; only the Researcher hits the read-only replica).

User Class	Presentation	UI Management	Application Logic	Data / Persistence
Prescribing Clinician	Submit variant lookup via web UI	Authenticate + check clinician role	QueryService facade: rank + role-redact	Use DB + in-memory cache
Clinical Pharmacist	Drill into evidence via provenance UI	Authenticate + check pharmacist role	QueryService + EvidenceService	Live DB + EvidenceRecord + PubMed cache
Bioinformatics Curator	Open curator dashboard; trigger refresh	Authenticate + check curator role	IngestionPipeline: fetch → validate → hold → check → promote	Holding-area schema then atomic swap into live DB
Researcher	Submit aggregate cohort query	Authenticate + check researcher role	GeneQueryStrategy + cohort-size guard	Read-only DB replica

3.3 Architecture-NFR Mapping

Each non-functional requirement is realized by a specific architectural mechanism. The traceability between Section 2.6 (the NFRs) and Section 3.2 (the architectural choices) is below.

Performance	Caching + indexed lookups → fast queries.
Security	Login at the API + redaction in the application logic → data is protected before it leaves the system.
Maintainability	Repository pattern + holding-area copy → monthly refreshes without disturbing live data.
Reliability	Pipe-and-filter pipeline with rollback → data refresh either fully succeeds or is fully reverted.
Interoperability	API-first design with auto-generated docs → other systems can integrate without bespoke contracts.

4 Implementation Details

4.1 Object-Oriented Design Process

Sommerville lays out a five-stage object-oriented design process. The proposal follows it directly:

1. Define the context and modes of use—covered in Sections 2.3, 2.5, and 2.7.
2. Design the system architecture—Section 3.
3. Identify the principal system objects—Section 4.2.
4. Develop design models—Sections 2.9 and 2.10.
5. Specify object interfaces—Section 4.3.

4.2 Principal System Objects

The principal system objects break down into four groups:

- **Domain entities:** *Variant*, *Gene*, *Drug*, *ClinicalAnnotation*, *EvidenceRecord*
- **Service objects:** *QueryService* (Facade), *IngestionPipeline*, *EvidenceService*, *AuditService*
- **Strategy handlers:** *VariantQueryStrategy*, *DrugQueryStrategy*, *GeneQueryStrategy* (one per query type, sharing a common *QueryStrategy* interface).
- **Repository objects:** one per domain entity (*VariantRepository*, *DrugRepository*, *ClinicalAnnotationRepository*, *EvidenceRecordRepository*, *AuditEntryRepository*).
- **Cross-cutting infrastructure:** *AuthMiddleware*, *RoleRedactor*, *CacheManager*

4.3 Object Interfaces

Three principal object interfaces are sketched here in plain language.

4.3.1 QueryService (Facade)

Three look-up methods, one per query type: *byVariant*, *byDrug*, *byGene*. Each takes a list of identifiers, an optional filter object, and a user-context object (carrying authentication and role information), and returns a structured *QueryResult*.

- Pre-condition: The user is authenticated and the current data release is loaded.
- Post-condition: An audit entry is written; the result respects the user's role-based redactions.

4.3.2 AnnotationRepository

Three find-by methods (*findByVariant*, *findByDrug*, *findByGene*). All three are indexed on the join keys for performance.

4.3.3 IngestionPipeline

The data-refresh pipeline, exposed as five filter operations executed in order:

- 1. *fetch (release_tag)*—pull source files from PharmGKB and DrugBank.
- 2. *validate (bundle)*—schema check.
- 3. *hold (bundle)*—write to the holding-area copy of the database.
- 4. *promote (release)*—atomic swap into live data.
- 5. *rollback (release_tag)*—abort and revert if anything fails.

Each filter is single-purpose and idempotent, and that's what makes rollback safe.

4.4 Design Patterns

Six design patterns appear in the design. Five of them are GoF design patterns from Sommerville; the sixth—Repository—is an architectural pattern, included here for completeness because it sits adjacent to the design-pattern decisions.

Pattern	Where used	Why
Repository	Between application logic and the database; one per entity	Decouples application from the database engine; alternative storage (e.g., in-memory for tests) needs no application changes
Strategy	One handler per query type (variant, drug, gene)	The three query types differ in indexing and ranking; Strategy isolates those differences
Facade	<i>QueryService</i> aggregates strategies, repositories, and the redactor	One entry point for controllers and any future client; simpler API surface
Observer	<i>IngestionPipeline</i> emits events on <i>promote</i> and <i>rollback</i> ; <i>AuditService</i> and <i>CacheManager</i> subscribe	Decouples data refresh from downstream concerns; new subscribers can be added without modifying the pipeline
Factory Method	<i>EvidenceRecord</i> .from_source("CPI C")	
Singleton	<i>CacheManager</i> and <i>AuthMiddleware</i> module-level instances	Cross-cutting infrastructure; exactly one instance per process

4.5 Implementation Issues

Sommerville frames implementation issues as three families—reuse, configuration management, and host-target development—and treats open-source development as a separate but adjacent concern. The proposal addresses all four.

4.5.1 Reuse

The proposal takes a hybrid reuse approach (which I think fits this project well). Open-source COTS components handle the commodity layers—web framework, ORM, database engine, cache, container tooling, authentication libraries—all under permissive licenses (MIT, BSD, or PostgreSQL). Custom build is reserved for the domain-specific logic: variant-name normalisation (with HGVS validation) and the evidence-ranking scoring engine that weights the two guideline bodies. Application-system reuse of a general Laboratory Information Management System was considered and rejected: the gating and ranking logic is too pharmacogenomics-specific to be off-the-shelf.

4.5.2 Configuration management

Git/GitHub flow with semantic versioning on tags. Continuous integration via GitHub Actions runs pytest, ruff (linting), and mypy (type checking) on every push. One detail worth highlighting: data-release tags are tracked alongside code tags in a catalog table, so any combination of (code version, data release) can be reproduced. Schema migrations are tooled—the database schema is never modified by hand.

4.5.3 Host-target development

Development is on Windows or macOS workstations; the target is a Linux container. Docker Compose pins the OS, Python runtime, database engine, Node.js runtime, and library versions, so the system runs identically across host and target.

4.5.4 Open-source development

The project's own code is released under the MIT license (permissive). Derived data from PharmGKB preserves the share-alike (CC-BY-SA 4.0) obligation. DrugBank's academic license is honored through role-based redaction at the API boundary: license-restricted fields are visible only to authenticated researchers and pharmacists whose institutional access is established.

4.6 Risks and Mitigations

Five risks, with concrete mitigations.

Risk	Mitigation
DrugBank academic-license ambiguity	Role-based redaction at the API boundary; legal review prior to deployment
Identifier mismatch between PharmGKB and DrugBank	Public drug-name terminology service (RxNorm) as the bridge; curator-visible unresolved set
Variant-naming differences across sources	Normalization layer with an HGVS validator and a golden-set of test cases

Stale data between releases	Scheduled monthly data refresh with a visible <i>last-refreshed</i> badge in the UI
Scope creep into hospital EHR integration	Explicitly deferred to Phase 4 and documented in the project roadmap

4.7 Documentation Plan

The proposal commits to a complete documentation set.

Product documents: Product Requirements Document (PRD), System Architecture Document (SAD), Technical Design Document (TDD), source-code documentation (generated from inline docstrings via Sphinx), API documentation (auto-generated from FastAPI), end-user manual, administrator runbook.

Process documents: Agile product roadmap and sprint backlogs, coding standards, test strategy, release checklist, working papers.

4.8 Roadmap

Development is phased over five increments.

Phase	Scope
Phase 0	This proposal—requirements, models, architecture, stack selection
Phase 1	MVP—data-refresh pipeline, read-only API, minimal user interface; enough to demonstrate one working CYP2C19-based query
Phase 2	Full system—authentication, audit logging, all three query types
Phase 3	Hardening—dashboards, exports, periodic CPIC and DPWG synchronisation
Phase 4	Deferred—direct EHR integration via FHIR Genomics, ML-based novel evidence ranking, multi-tenant deployment

ACKNOWLEDGMENTS

The author wishes to thank Jan Michael C. Yap, Ph.D., for the Bioinfo 203 lectures and reference materials that anchored the requirements, modelling, and architectural framing of this proposal, and for guidance throughout the semester. Thanks to classmates as well for the thoughtful questions and constructive feedback in the online forum. Finally, the author acknowledges the open-data and open-source communities behind PharmGKB, DrugBank, CPIC, DPWG, PubMed, and the supporting tooling stack—without their freely accessible curated resources, a proposal of this scope would not be feasible.

REFERENCES

[1] Sommerville, I. *Software Engineering*, 10th edition. Pearson, 2014.

[2] Pirmohamed, M., James, S., Meakin, S., Green, C., Scott, A. K., Walley, T. J., Farrar, K., Park, B. K., and Breckenridge, A. M. "Adverse drug reactions as cause of admission to hospital: prospective analysis of 18,820 patients." *British Medical Journal*, 329(7456): 15–19, 2004. <https://doi.org/10.1136/bmj.329.7456.15>

[3] Whirl-Carrillo, M., Huddart, R., Gong, L., Sangkuhl, K., Thorn, C. F., Whaley, R., and Klein, T. E. "An Evidence-Based Framework for Evaluating Pharmacogenomics Knowledge for Personalized Medicine." *Clinical Pharmacology & Therapeutics*, 110(3): 563–572, 2021. (PharmGKB). <https://doi.org/10.1002/cpt.2350>

[4] Wishart, D. S., Feunang, Y. D., Guo, A. C., Lo, E. J., Marcu, A., Grant, J. R., Sajed, T., Johnson, D., Li, C., Sayeeda, Z., Assempour, N., Iynkkaran, I., Liu, Y., Maciejewski, A., Gale, N., Wilson, A., Chin, L., Cummings, R., Le, D., Pon, A., Knox, C., and Wilson, M. "DrugBank 5.0: a major update to the DrugBank database for 2018." *Nucleic Acids Research*, 46(D1): D1074–D1082, 2018. <https://doi.org/10.1093/nar/gkx1037>

[5] Relling, M. V., and Klein, T. E. "CPIC: Clinical Pharmacogenetics Implementation Consortium of the Pharmacogenomics Research Network." *Clinical Pharmacology & Therapeutics*, 89(3): 464–467, 2011. <https://doi.org/10.1038/clpt.2010.279>

[6] Swen, J. J., Nijenhuis, M., de Boer, A., Grandia, L., Maitland-van der Zee, A. H., Mulder, H., Rongen, G. A. P. J. M., van Schaik, R. H. N., Schalekamp, T., Touw, D. J., van der Weide, J., Wilffert, B., Deneer, V. H. M., and Guchelaar, H.-J. "Pharmacogenetics: from bench to byte—an update of guidelines." *Clinical Pharmacology & Therapeutics*, 89(5): 662–673, 2011. (DPWG). <https://doi.org/10.1038/clpt.2011.34>

[7] U.S. Food and Drug Administration. *Table of Pharmacogenomic Biomarkers in Drug Labeling*. <https://www.fda.gov/drugs/science-and-research-drugs/table-pharmacogenomic-biomarkers-drug-labeling>

[8] HL7 International. *FHIR Genomics Implementation Guide*. <https://hl7.org/fhir/genomics.html>

[9] den Dunnen, J. T., Dalgleish, R., Maglott, D. R., Hart, R. K., Greenblatt, M. S., McGowan-Jordan, J., Roux, A.-F., Smith, T., Antonarakis, S. E., and Taschner, P. E. M. "HGVS Recommendations for the Description of Sequence Variants: 2016 Update." *Human Mutation*, 37(6): 564–569, 2016. <https://doi.org/10.1002/humu.22981>

[10] Tiangolo, S. *FastAPI: A Modern, Fast (high-performance) Web Framework for Building APIs*. Project documentation. <https://fastapi.tiangolo.com/>

[11] Bayer, M. *SQLAlchemy 2.0 Documentation*. <https://www.sqlalchemy.org/>

[12] Yap, J. M. C. *Bioinfo 203 Lecture Slides—Sessions 1-6*. Department of Computer Science, University of the Philippines Diliman, AY 2025–2026.